DAY 2 · SESSION 1

Top 10 Key Takeaways

Distilled from the diarized transcript, Chunks 1–9 (~13:20–14:46). Instructors: Madeleine (MW) and Marlon (MK), with Becca on Git and faculty guests Jung Yoon and Natasha on real projects.

1. **"Claude Code" is a misnomer — treat it as the most powerful way to use the LLM, not a coding tool.** MK's framing: despite the name, it's an *authoring environment* for any output — academic research, course materials, writing of all kinds. The point of the day is that Code is "the most sophisticated and powerful way of interacting with Claude," code or no code.

2. **Two on-ramps, no one left behind: desktop app vs. IDE/terminal.** Non-developers stay in the **Code panel of the desktop app** (Mac users just click in; Windows users pay a "tax" — *git* + virtualization, with staff help). Developers comfortable with a terminal set up the **CLI / VS Code (IDE)** path (~15 min of installs). MW and MK deliberately **alternated interfaces** to show the *same operations* in both.

3. **A GitHub repo is "just a bundle of files" — Google Drive for code.** Everything was distributed as a Git repo (date-coded *claude-code-20260602*). Two ways in: **download the green "Code → zip"** button (when in doubt, grab the zip), or **clone via HTTPS**. MK's desktop-app trick: paste the URL and literally ask Claude *"can you clone this repository for me?"* — then grant permission.

4. **Point Claude at one dedicated folder — never your whole desktop.** MK created a fresh *MK today* folder to clone into, with the explicit warning: don't hand Claude access to everything ("my desktop's got a lot of stuff I don't want it to see"). The recurring discipline from Day 1 — *one specific folder* — carries straight into Code.

5. **The repo mirrors the recipe: inputs / prompts / outputs.** The tour walked the file structure — *my-recipe* (your ingredients, prompts, tools, then outputs), stock *learning-lab-examples*, *faculty-recipes* built from attendees' recipe cards, plus *resources* (glossary, handouts, checklists, the Day 1 recap). Same ingredients → instructions → dish mental model, now as folders on disk.

DAY 2 · SESSION 1 (CONT.)

Takeaways 6–10

CLAUDE.md, Git for everyone, context windows at scale.

6. **The mystery limerick = proof that one line of text rules an entire folder.** Every answer in the repo kept ending in a limerick ("The model's not magic, just text…"). The reveal: a **CLAUDE.md at the repo root** with one silly instruction — *always end with a limerick*. MW's payoff: "This is just one line of text I put into a markdown document. That is how influential text is in these systems."

7. **CLAUDE.md is an automatic, project-level system prompt.** It gets read in *every time* you work in that folder — on top of Claude's own system prompt and memory. MK edited it live (limerick → rhyming couplet), with Git's diff showing red removed / green added. Quirks: **save the file**, and **open a new session** for it to load (in the folder that holds the CLAUDE.md). Bonus: Claude walks *up* the folder tree to your home directory, reading every CLAUDE.md along the way.

8. **Git is for everyone who writes anything — Becca's four reasons.** Not just code. (1) **Protects against loss/damage** — roll back when Claude "messed up this file"; a private remote doubles as backup. (2) **Claude deeply understands Git** — it reconstructs what a project is about from commit history, getting around the context-window limit across sessions. (3) **Collaboration** — branches let multiple people (or Madeleine's "ten-windows-out-of-spite" Claude sessions) work without overwriting each other. Core moves: clone / init / **commit often** / push / pull request.

9. **Two safety rails Becca flagged: pull requests and *.gitignore*.** Code's interface nudges you toward **"create a pull request"** — work on a copy/branch, then merge into main — good practice even solo. And **.gitignore** keeps secrets (API keys, passwords, proprietary docs, huge files) out of a shared repo; ask Claude to make one and it will.

10. **Context windows are the same everywhere — and Code is where you spend them at scale.** Every chat / Cowork / Code session is a context window; */context* shows exactly what's loaded and how many of your 1M tokens are spent. Two ways to feed files: let Claude **search** (burns tokens) or **copy the exact path** (precise, cheap). MW's Finn McCool / Natasha example: PDFs, audio transcripts, scans, and book chapters become — *in an afternoon* — a whole interrelated system of HTML files. That's the "superpower," and "it can only happen in Claude Code."

SECONDARY POINTS WORTH KEEPING

- **The day's running prompt:** *"I missed the session yesterday — what are 10 things I should know? Here's the path to the transcript."* The summarizer pattern itself was the running demo.
- **Two ways to give Claude a file,** restated: search vs. copy-path. Harvard free accounts have a daily token ceiling, so reckless searching wastes your budget — be direct.
- **Find your files in the desktop app:** the file-browser icon (top right) only appears *after* you've started a chat; click it → Files.
- **CLAUDE.md as a zero-code prototype:** the "question anticipation" faculty recipe became interactive **student personas** you can converse with — a hacky no-UI stand-in for a Slack app (teased for Day 4).
- **Tomorrow (Day 3):** more file types beyond CLAUDE.md — **skills** and the rest of the system that gets auto-read.
- **Tmux** lets power users run many Claude sessions at once across terminal panes; hooks "that sound like birds" announce what's happening.